B. Gaspard
B. Thomas
D. Hugo
G.O. Noam
P. Maxence

datsmysong

-

Rapport de fin de semestre

Tuteur : Florian Ferbach

Année scolaire : 2023 - 2024

Table des matières

Table des matières 2
Introduction 3
Analyse 4
Modification de notre planning de déroulement par rapport à l'étude préalable 5
Réalisation 7
Intégration de plusieurs plateformes (orienté objet) 7
Communication en temps réel 8
Difficultés rencontrés 10
Planning de déroulement du projet durant les 4 itérations de ce semestre 12
Première itération 12
Deuxième itération 12
Troisième itération 12
Quatrième itération 12
Répartition du travail entre étudiants 14
Éléments originaux dont nous sommes fiers 16
Thomas 16
Hugo 17
Noam 18
Maxence 19
Gaspard 21
Objectifs à atteindre pour la fin du projet. Planning pour les deux itérations restantes 23
Objectif à atteindre pour l'itération 6 23
Cinquième itération 23
Sixième itération 23
Conclusion 25

Introduction

Ce document est un rapport de fin de semestre concernant le projet tutoré datsmysong.

Ce projet consiste à la réalisation d'une application mobile et web dont l'objectif est de permettre une écoute musicale à partir d'une file d'attente de musique collaborative, à l'instar de la fonctionnalité Spotify fraîchement sortie appelée Spotify Jam[1]. Ce type d'application serait grandement utile lors de rassemblements festifs entre amis ou en famille afin que tout le monde puisse ajouter sa touche à l'ambiance musicale.

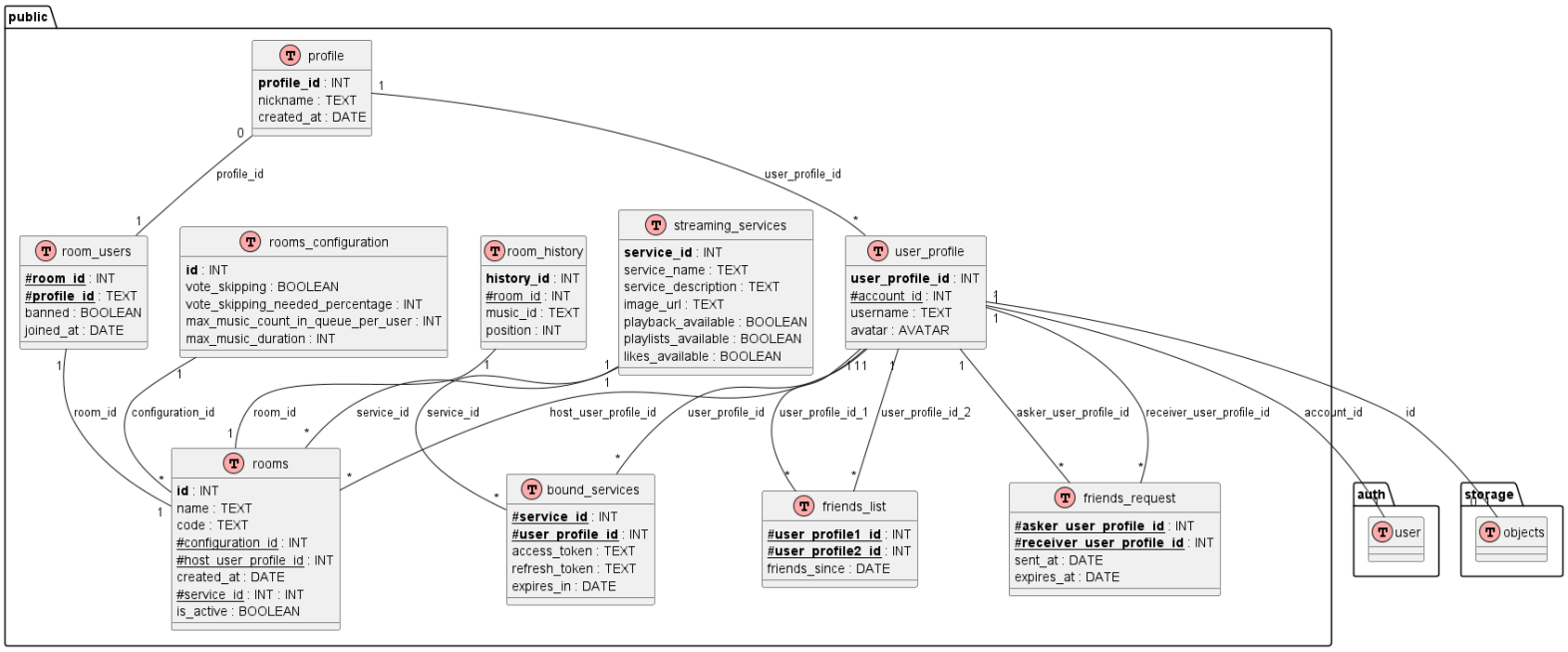
L'application doit pouvoir fonctionner avec différentes plateformes musicales, on parlera de « platform agnostic ». L'utilisation ne sera donc pas limitée à Spotify, ni à une autre plateforme.

Les utilisateurs sont regroupés dans des salles d'écoute dans lesquelles on dira qu'ils sont "participants", ces salles d'écoute sont créées par un utilisateur qu'on appellera "hôte", et chaque salle d'écoute comportera une file d'attente partagée avec les musiques que les participants auront ajouté.

[1] https://support.spotify.com/fr/article/jam/

Analyse

Notre base de données a subi de légères modifications par rapport au diagramme initial que nous avions imaginé. Nous avions lors de l'itération 1 créé une nouvelle table active_rooms, partagent en grande majorité le schéma de la table rooms, permettant ainsi de consulter les salles actives et inactives dans deux tables différentes, mais nous sommes revenu sur ce choix qui nous a forcé à dupliquer des données et a surtout complexifié notre schéma et par conséquent notre logique métier avec au bout du compte très peu de gain. Nous avons donc pris la décision lors de l'itération 4 de rassembler ces deux tables à nouveau.



Modification de notre planning de déroulement par rapport à l'étude préalable

Itération 1

- Créer un compte (seulement compte Spotify) (Gaspard)
- Création d'une salle d'écoute ~~(Maxence)~~ Thomas
- Générer un lien vers la salle d'écoute et rejoindre une salle d'écoute grâce à un lien (Noam)
- Ajout d'une musique possible uniquement avec un lien Spotify ~~(Thomas)~~ Hugo
- Consulter la file d'attente (Hugo)
- Créer la logique frontend des interactions avec les plateformes de musique (qui dépendent de la nature de la plateforme de streaming à intégrer, API ou Widget par exemple) (Maxence)
  - Créer le schéma de données à utiliser côté frontend

Itération 2

- Implémenter pour chaque plateforme de streaming les méthodes du lecteur audio (volume, play/pause, suivant/précédent, ajout en file d'attente)
  - Spotify ~~(Gaspard)~~
  - SoundCloud (Maxence)
- Créer un compte (adresse mail et mot de passe) ~~(Hugo)~~ Gaspard
- Lier son compte Spotify (Thomas)
- Consulter l'historique et les détails d'une salle d'écoute (participants, musiques jouées...) ~~(Noam)~~ Maxence

+ Rejoindre une salle d'écoute (page rejoindre) (Noam)

+ Authentification avec Google (Gaspard)

-

Itération 3

- Ajouter une musique (barre de recherche, lien d'autres plateformes...) (Maxence)
- Voter pour passer une musique (Gaspard)
- Supprimer une musique de la file d'attente (Noam)
- Paramétrer une salle d'écoute (vote, nombre de musiques max par utilisateur...) (Thomas ~~& Hugo~~)

Itération 4

- Rejoindre une salle d'écoute avec un petit code ou un QR code ~~(Gaspard)~~ Noam
- Fermer une salle d'écoute (hôte) (Hugo)
- Quitter une salle d'écoute (participant) (Noam)

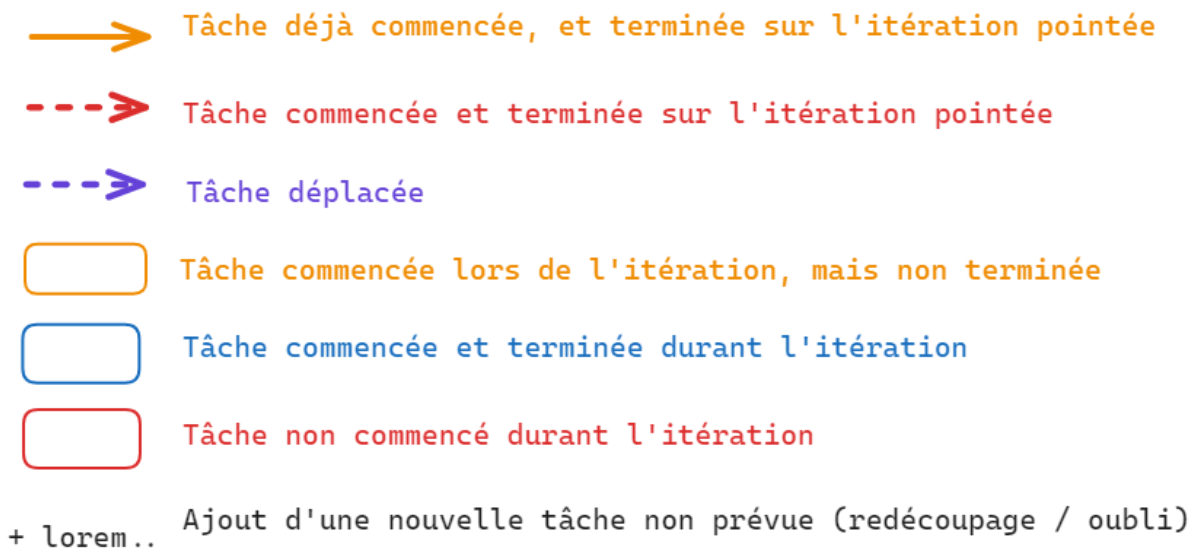
Itération 5

- Expulser/bannir un participant de la salle d'écoute (Thomas)
- Liaison du compte avec services populaires (Deezer, etc...) (Gaspard)
- Permettre à un participant d'accéder à ses playlists et coups de coeurs sur les plateformes de streaming liées (Spotify, Deezer, etc...) et d'ajouter ces musiques là dans la salle d'écoute (Maxence)

Commencé itération 3
Finalisé itération 4

Itération 6

- Rejoindre une salle d'écoute en tant qu'invité (via un lien) (Noam)
- Système d'amis (consulter, supprimer, demander, refuser) (Hugo)
- Modifier les paramètres de son profil (nom d'utilisateur, avatar, etc) ~~(Thomas)~~ Gaspard



Nous pouvons remarquer que notre planning à énormément changé. Dans un premier temps, nous n'avons pas respecté les personnes assignés sur les tâches. En effet, certaines tâches étaient plus longues que d'autres, nous nous sommes donc parfois replacés sur certaines tâches. Mais la chose importante que nous pouvons observer est le décalage de prêt de 3 tâches après l'itération 4, à cause de certains retards. Il était très difficile de savoir le temps que chaque tâche allait prendre, nous avons donc dû en décaler, commencer certaines en toute fin d'itération donc les terminer plus tard, mais l'ordre à était quasiment respecté.

Réalisation

Nous avons une architecture client/serveur relativement classique, avec un backend TypeScript utilisant le framework Fastify (alternative plus performante et légère à ExpressJS), et un client React Native utilisant le framework Expo et de nombreux modules.
En ce qui concerne notre base de données, nous utilisons le service Supabase, qui héberge une base de données PostgreSQL, fournit un SDK TypeScript avec une API moderne et dont l'expérience développeur est inégalée, ce qui nous permet de rapidement avancer.
Nous n'avons pas de test E2E ou unitaires, car créer des tests sur un projet Web (et qui plus est, natif) est une tâche lourde demandant énormément de temps, ce qui n'était pas compatible avec les contraintes de temps que nous avons.
En revanche, nous avons des configurations Prettier et ESLint permettant d'obtenir des avertissements voire des erreurs dès que la qualité du code n'atteint pas les règles définies, et des pipelines CI vérifiant le bon respect de ces règles à chaque PR ouverte, permettant ainsi de s'assurer de la bonne qualité du code.

Intégration de plusieurs plateformes (orienté objet)

L'un des plus gros enjeux de notre projet était l'intégration de plusieurs plateformes musicales. Lors de l'étude préalable, nous avions analysé les 5 plateformes musicales connues afin de nous concentrer sur celles permettant une intégration de notre application. Spotify et Apple Music étaient les deux plateformes permettant une intégration de notre application avec le lecteur appareil de l'utilisateur, mais Apple Music étant payant, nous n'avons pas pu le retenir. Nous devions donc trouver une autre plateforme afin de commencer avec 2 plateformes différentes. Nous nous sommes donc intéressés à SoundCloud, qui ne possède pas d'API pour gérer la lecture sur le compte de l'utilisateur, mais qui nous laisse la possibilité de créer un lecteur sur l'une de notre page Web à l'aide d'un Widget.
Pour résumer, nous avons déterminé deux types de plateformes pour la lecture de notre musique :

- lecteur de musique locale
- lecteur de musique distante

L'enjeu était de se concentrer sur une implémentation réfléchie pour que les deux types de lecture soient gérés de la même manière côté client au niveau des contrôles du lecteur (pause, play, suivant, etc), et que nous gardions la possibilité d'ajouter d'autres services par la suite tels qu'Apple Music, sans devoir modifier toute l'architecture de notre application.

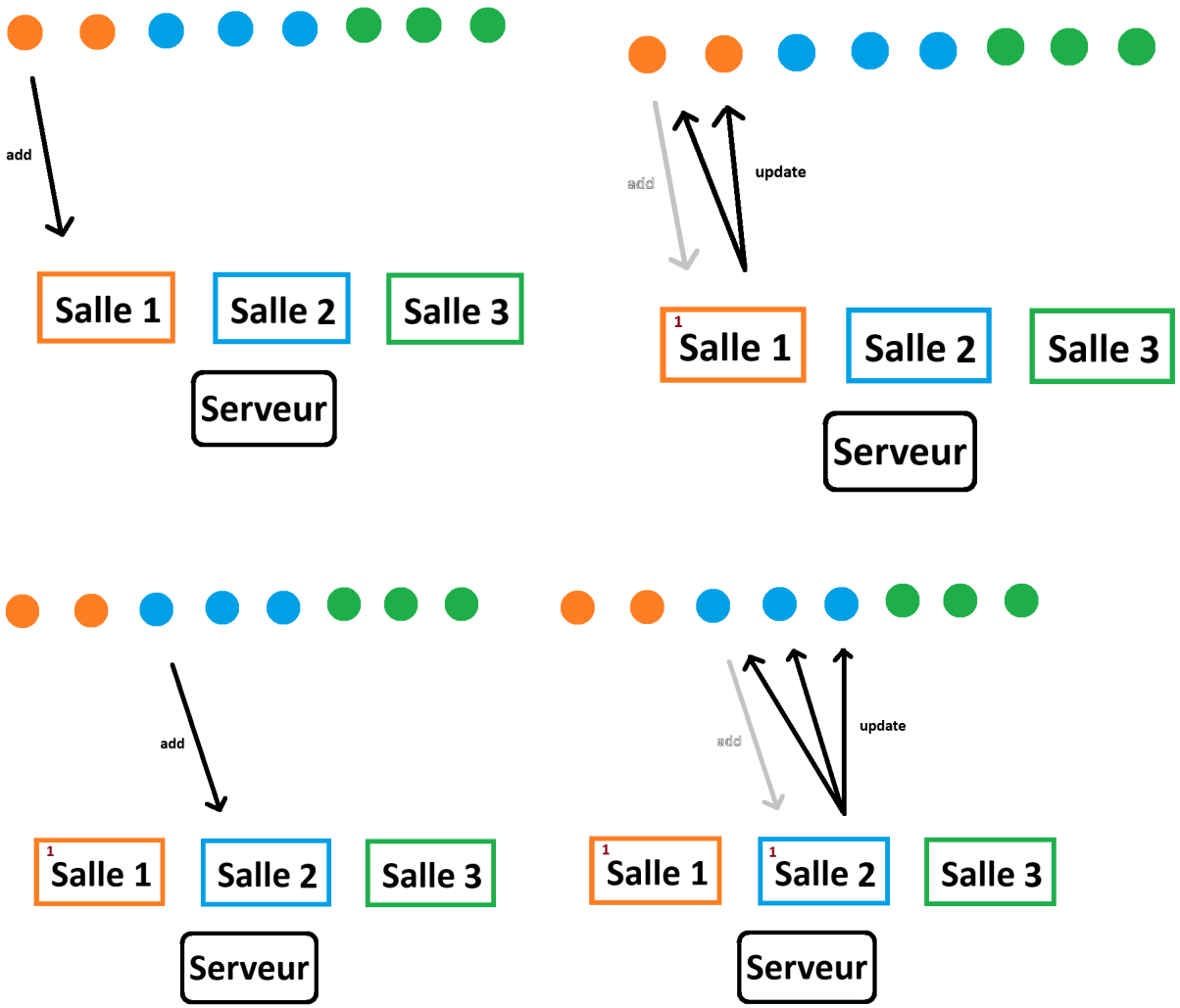
En lien avec la partie d'après sur les Websocket, nous devions implémenter une communication en temps réel entre les utilisateurs dans une salle active pour que l'état le la lecture ainsi que la file d'attente par exemple soient mis à jour pour chacun dès que nécessaire, c'est-à-dire en temps réel. Le moyen le plus simple est de passer par un serveur central. Afin d'implémenter de la même manière le Websocket que l'on soit sur un lecteur de musique locale ou distant, nous avons utilisé une approche orientée objet en TypeScript, avec une interface étant implémentée par chaque service et le serveur pourra simplement gérer en appelant sur un objet du type de notre interface.
Maxence décrit plus précisément cela dans la partie de l'explication de l'élément dont il est fier dans notre projet.

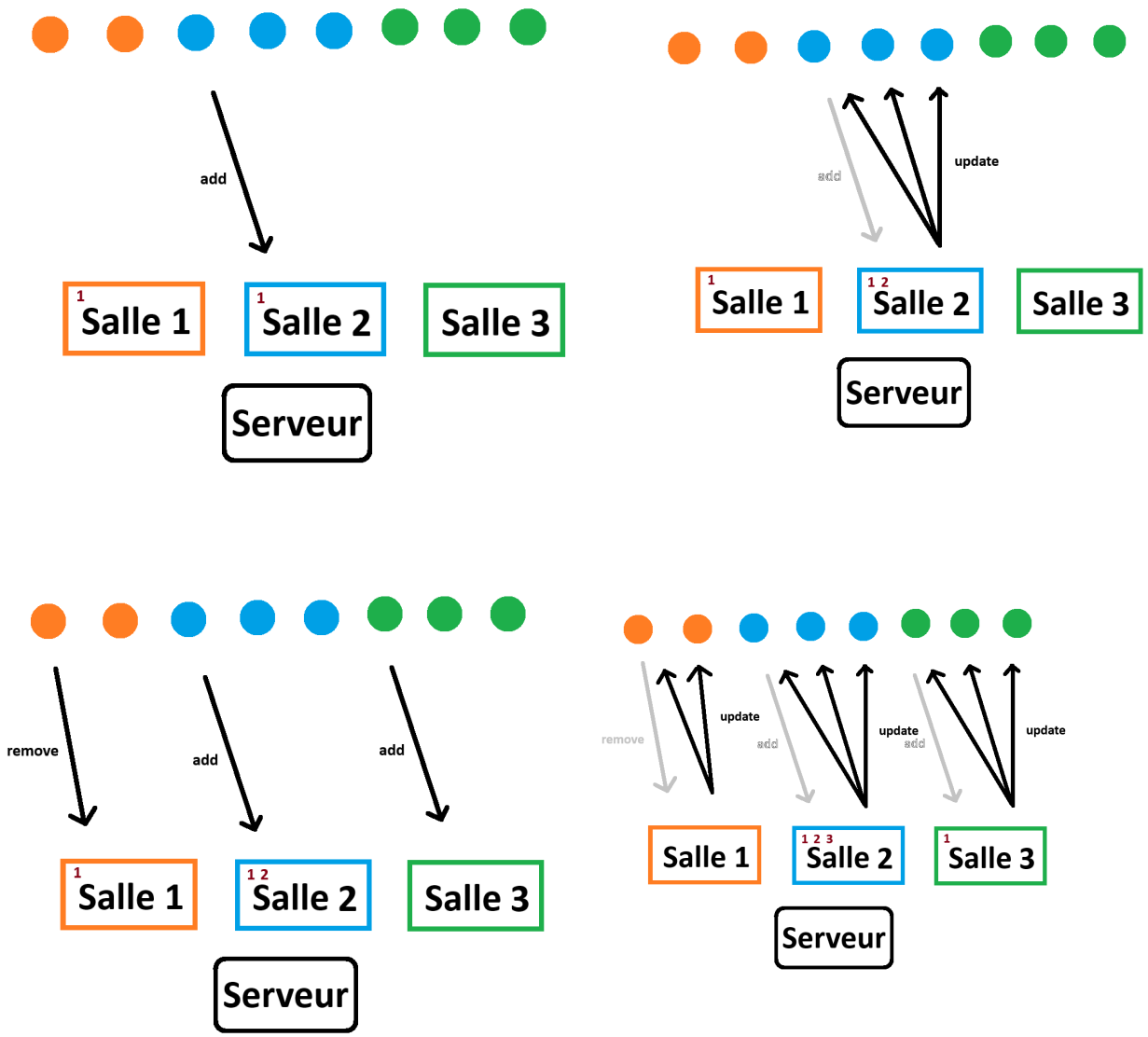
Communication en temps réel

Lors de notre analyse, nous avions déclaré comme risque la communication en temps réel et la synchronisation entre les différents appareils. C'est un élément fondamental de notre application et dès la première itération, Hugo s'est lancé dans cette tâche. Cette tâche a pris quasiment 3 itérations à être terminée, car elle était assez complexe, et nous avons pris le temps de faire une bonne architecture pour la communication en temps réel. En effet, la tâche d'Hugo ne comportait que consulter la liste de lecture, mais les WebSocket n'étant pas encore implémentés dans l'application, c'est à lui qu'est revenue la tâche de réaliser l'infrastructure de code nécessaire pour les supporter. Par la suite, Maxence a dû attendre une itération complète en mettant en pause ses deux premières tâches, nécessitant cette communication en temps réel pour avancer.

Pour plus de détails, consultez l'élément dont Hugo est fier dans notre projet.

Schémas d'implémentation :





Difficultés rencontrés

La difficulté que nous avons tous éprouvée est le ralentissement lié aux technologies utilisées. En effet, tous les membres n'étaient pas au même niveau de familiarité avec les outils que nous avons décidé d'utiliser. Pour surmonter cet obstacle, nous avons mis en place une stratégie *efficace* en assignant des tâches en fonction des compétences individuelles et de l'appétence de chacun envers chacune des tâches. Cette approche a permis de maximiser l'efficacité du groupe, car chacun pouvait contribuer de manière significative en fonction de ses connaissances et de son expérience. Malgré les difficultés initiales, la découverte de ces

technologies innovantes nous a non seulement permis de renforcer nos compétences individuelles, mais aussi de créer un environnement d'apprentissage collaboratif au sein de l'équipe. Nous avons dès le début instauré des *code review* où chacun devait consulter et commenter le code des autres, dans le but de lever de potentiels problèmes et d'élever la qualité du code. A partir de la 3ième itération, nous avons appliqué notre nouveau workflow suggéré par notre tuteur, qui contenait une heure de code review dédiée à partir de 17h, permettant ainsi de consulter le code écrit la journée par nos camarades.

Planning de déroulement du projet durant les 4 itérations de ce semestre.

Voici le résumé des tâches terminés sur la branche main, à chaque fin d'itération

Note: chaque itération comporte uniquement les tâches terminées, il se peut donc qu'un membre du groupe n'apparaisse pas, dans le cas où il n'aurait pas fini une tâche assignée.

Première itération

- Créer un compte utilisateur à partir d'un compte Spotify et se connecter
- Créer une salle d'écoute
- Création et intégration d'un composant Bouton

Deuxième itération

- Créer un compte utilisateur à partir d'un compte Google
- Suppression des bugs liés à l'authentification
- Créer les maquettes nécessaires pour la page des intégrations

Troisième itération

- Générer un lien vers une salle d'écoute et rejoindre une salle d'écoute grâce à ce lien
- Création de compte à partir d'une adresse email et d'un mot de passe
- Modification du composant Bouton afin d'y ajouter une propriété de couleur

Quatrième itération

- Créer la logique des interactions avec les plateformes de musique
- Implémenter la logique pour chaque plateformes de streaming
- Menu d'intégrations des plateformes (Spotify et Soundcloud implémenté)
- Modifier les paramètres de son profil (nom, email, avatar stocké en base de donnée)
- Fermeture d'une salle d'écoute pour le créateur de la salle
- Suppression d'une musique pour le créateur de la salle

- Ajouter une musique à la file d'attente (backend)
- Consulter la file d'attente d'une salle d'écoute
- Afficher un QR Code sur un écran de partage
- Rejoindre une salle d’écoute à l’aide d’un code

Répartition du travail entre étudiants

Gaspard :

- Créer un compte utilisateur à partir d'un compte Spotify et se connecter
- Créer un compte utilisateur à partir d'un compte Google
- Suppression des bugs liés à l'authentification
- Création de compte à partir d'une adresse email et d'un mot de passe
- Modifier les paramètres de son profil (nom, email, avatar stocké en base de donnée)
- Voter pour passer une musique (en cours, ou en file d'attente)

Noam :

- Générer un lien vers une salle d'écoute et rejoindre une salle d'écoute grâce à ce lien
- Rejoindre une salle d'écoute à l'aide d'un code
- Interface pour rejoindre une salle d'écoute à l'aide d'un code
- Afficher un QR Code sur un écran de partage
- Suppression d'une musique pour le créateur de la salle

Maxence :

- Créer la logique des interactions avec les plateformes de musique
- Implémenter la logique pour chaque plateformes de streaming
- Créer les maquettes nécessaires pour la page des intégrations
- Création et intégration d'un composant Bouton
- Créer la logique des interactions avec les plateformes de musique
- Implémenter la logique pour chaque plateforme de streaming

Hugo :

- Ajouter une musique à la file d'attente (backend)
- Consulter la file d'attente d'une salle d'écoute
- Fermeture d'une salle d'écoute pour le créateur de la salle

Thomas :

- Créer une salle d'écoute
- Suppression d'un bug sur le backend pour la route permettant la création de room
- Menu d'intégrations des plateformes (Spotify et Soundcloud implémenté)
- Créer le menu de paramétrage d'une salle d'écoute

Pour répartir les tâches, nous avons pris en compte l'expérience en programmation de chaque membre du groupe et la difficulté de chaque tâche. De cette manière, les tâches les plus difficiles allaient aux membres les plus expérimentés et les tâches les plus faciles aux membres les moins expérimentés.

Éléments originaux dont nous sommes fiers

Thomas

Je pense que la tâche dont je suis le plus fier à la fin de ces 4 itérations, est celle que j'ai réalisée en dernier : la configuration d'une salle d'écoute. Cette tâche est accessible depuis une salle d'écoute, seulement quand l'utilisateur est l'hôte de la salle d'écoute. Pour cette page j'ai dû utiliser des sliders… Et le composant slider de react native est deprecated, j'ai dû utiliser un slider venant des composants de la communauté. D'après mes recherches c'était le meilleur composant slider qui existe, j'ai donc décidé de le prendre. Néanmoins, malgré le fait que c'est censé être le meilleur composant slider existant, après le début de mon implémentation, je me suis rendu compte qu'il manquait beaucoup de choses à ce composants. Par exemple, on ne peut pas changer la taille du pouce du curseur ou encore pouvoir mettre une étiquette pour avoir la valeur du slider en direct.
Pour pouvoir résoudre ces problèmes, il fallait que je trouve une solution pour pouvoir faire moi même ce que le composant ne faisait. Pour pouvoir avoir une étiquette qui suit le pouce du curseur. J'ai moi même mis un texte avec un fond noir au dessus du slider, j'ai ensuite rajouté un triangle (image exporté de figma) en dessous pour donner quelque chose qui ressemble à ça :

Pour l'instant, j'avais seulement cela, avec la valeur qui changeait à chaque fois que l'on bougeait le slider, ce qu'il me restait à faire était que tout cela bouge pour être pile au-dessus du pouce du curseur. Pour cela, j'ai dû ajouter une propriété CSS : left, et la modifier en fonction de la valeur du curseur. Pendant plusieurs heures nous avons cherché avec Hugo le calcul qui permettrait d'aligner le label et le curseur. Pour arriver à cette formule :
(ValeurActuelle - ValeurMin) * ((TailleSlider - TaillePouce) / (ValeurMax - ValeurMin) - 6
L'étape suivante est de changer la taille du pouce du curseur, pour cela il faut que j'exporte celui qui était dans figma, et que à la place de curseur directement dans le composant slider, je

met l'image exportée de figma, qui est en réalité seulement un petit point noir, néanmoins c'est ce que je pensais… Car cela ne changeait en aucun cas la taille du curseur, et après des recherches je n'ai pas trouvé de moyens de changer la taille du curseur. J'ai donc décidé de laisser le curseur à sa taille de base.

Depuis le début, je parle de sliders, néanmoins, il n'y a pas que des sliders dans cette page. Car cette page permet de changer tous les paramètres d'une salle d'écoute, j'ai donc utilisé des checkboxs et des textInput. J'avais déjà implémenté ces deux éléments dans une autre page, j'ai donc seulement eu à reprendre ce que j'avais déjà fait, ce n'est donc pas très intéressant. Néanmoins ce qu'il me restait à faire est de lier le frontend et le backend. Dans le backend, j'ai au début de cette tâche, fait une route dans notre API qui permettait de modifier une configuration de salle dans la table room_configuration. Pour finir, j'ai mis des événements sur les éléments de la page afin que quand on change les valeurs, elles sont aussi changées dans la BD.

Hugo

J'ai implémenté un système de communication bilatéral appelé socket.io, qui permet de faire communiquer un serveur avec des clients, en l'occurrence ici, cela représente des navigateurs internet ou des applications mobiles de l'utilisateur. C'est-à-dire que les deux parties, client ou serveur, peuvent s'envoyer des données diverses à tout moment, ainsi l'autre partie, qui reçoit les données, peut les récupérer à l'aide d'un système d'événement. Quand un utilisateur rejoint une salle d'écoute, l'application se connecte au serveur socket.io avec un chemin contenant l'identifiant de la salle d'écoute permettant au serveur de comprendre quelle salle d'écoute l'utilisateur rejoint et ainsi adapter le contenu des données qui sera transmis dans le futur.

En effet, à chaque modification de la file d'attente de la salle d'écoute, le serveur transmet la salle d'écoute complète à tous les utilisateurs. Ensuite, les clients se chargent de mettre à jour la partie graphique en affichant la salle d'écoute mise à jour. Dans le futur, nous pourrions imaginer une mise à jour partielle de la file d'attente, c'est-à-dire communiquer au clients uniquement les parties modifiées. C'est le principe Publish-Subscribe ou publier-s'abonner en français, en effet, les utilisateurs "s'abonnent" à une salle d'écoute, et ainsi reçoivent toutes

les notifications concernant la salle d'écoute. De plus, cela leur permet aussi de "publier" des modifications sur la salle d'écoute comme l'ajout ou la suppression de morceaux.
Socket.io est une bibliothèque permettant créer une connexion bilatérale entre client et serveur de façon simple, il s'adapte aux différentes possibilités de connexion du client : websocket si disponible, ou requête HTTP.

Noam

L'élément dont je suis le plus fier est probablement la génération d'un lien d'invitation vers une salle d'écoute, ainsi que la façon dont est géré ce lien d'invitation.
Lorsque l'utilisateur est dans une salle d'écoute, il peut accéder à un menu d'invitation, dans lequel il peut copier le fameux lien d'invitation en cliquant sur un bouton. Lorsque celui-ci le partage à un autre utilisateur, le lien agit de différente façon en fonction du cas. S'il est connecté, la page demande à l'utilisateur s'il veut continuer sur le site ou ouvrir la page de la salle d'écoute dans l'application. La page agit de la même façon, qu'il soit déjà participant ou non, la seule différence est que s'il ne l'est pas, il est ajouté aux participants dans la base de données. En revanche, s'il n'est pas connecté, il peut soit se connecter, soit continuer en tant qu'invité (ce qui est, à l'heure, non implémenté). Bien évidemment, le choix de la plateforme n'est pas valable pour les utilisateurs PC, car ils n'ont pas accès à l'application mobile.
C'était la tâche avec laquelle j'ai commencé le projet et avec laquelle j'ai appris à utiliser Expo/React-Native, et même si le code final n'était pas des plus qualitatif, elle m'a beaucoup appris sur les frameworks.

Maxence

L'élément dont je suis le plus fier est la fonctionnalité de lecture des musiques que j'ai commencé à l'itération 1 et qui est enfin finalisée à l'itération 4. C'est une partie relativement complexe de l'application étant donné que nous devons supporter différentes plateformes, dont le fonctionnement est fondamentalement différent. Pour Spotify, il suffit de faire des appels sur une API REST pour contrôler le lecteur de l'application Spotify, tandis que pour notre seconde plateforme pour le moment, SoundCloud, il faut charger un Widget sur le client de l'hôte, et contrôler la lecture en local donc. Pour finaliser ma tâche j'ai dû attendre la finalisation d'une tâche d'Hugo portant sur les websockets, qui a été finalisée dimanche 28 janvier. Le fonctionnement côté client est relativement simple sur la surface: on clique sur le bouton de lecture, un message est envoyé au serveur dans le websocket ouvert qui indique que l'hôte souhaite lancer la lecture, on clique sur le bouton de pause, un message est envoyé au serveur dans le websocket ouvert qui indique que l'hôte souhaite mettre en pause la lecture, et ainsi de suite. En revanche, côté serveur, le fonctionnement diffère en fonction de la plateforme utilisée, et cela est régi par un système d'héritage de télécommande. Une interface de télécommande du lecteur de musique doit être implémentée pour chaque plateforme de streaming. Pour Spotify, on utilise le SDK TypeScript fourni par Spotify pour gérer le lecteur à partir des tokens de l'hôte stockés dans notre base de données, ce fonctionnement étant relativement simple. Pour SoundCloud en revanche, c'est une autre histoire, puisqu'on renvoie dans le websocket connecté à l'hôte un message indiquant de mettre la lecture en pause, ou de changer le volume, ou de lancer une musique précise, etc… Ce message est ensuite reçu par le client via le tunnel websocket, qui, lorsque le lecteur de musique local est disponible (ce qui doit être le cas, puisque le serveur n'enverrait pas ce message autrement), effectue une action sur ce dernier. D'ailleurs, l'implémentation du widget SoundCloud est plutôt complexe, puisque le widget est chargé dans un iframe, mais pour contrôler cet iframe, un SDK JavaScript est fourni, qui doit donc être chargé, sauf qu'il est impossible de le charger dans le contexte d'une application native. Ce qu'il se passe donc, c'est qu'un iframe/WebView est chargé dans l'application, qui contient une page web qui charge le widget SoundCloud ainsi que le SDK permettant de le contrôler. Pour contrôler ce widget, on passe par le système de message des iframes, avec les méthodes postMessage et l'événement message. L'iframe écoute pour les commandes, et effectue les actions nécessaires. Notre

application envoie un événement à notre iframe, qui communique avec l'iframe SoundCloud. Pour les commandes qui attendent un retour, comme celle pour récupérer l'état de la lecture, notre iframe renvoie un message à la fenêtre parent lorsqu'il reçoit l'état de lecture du widget SoundCloud, et nous écoutons pour ces événements dans notre composant SoundCloudPlayer. Pour permettre une utilisation optimale, les méthodes comme getPlaybackState(), permettant de justement récupérer l'état de la lecture, retourne une promesse, qui est résolue avec l'état de la lecture lorsque le listener de notre iframe reçoit l'événement correspondant.
Voici un schéma détaillant les intéractions avec les différents acteurs:

Gaspard

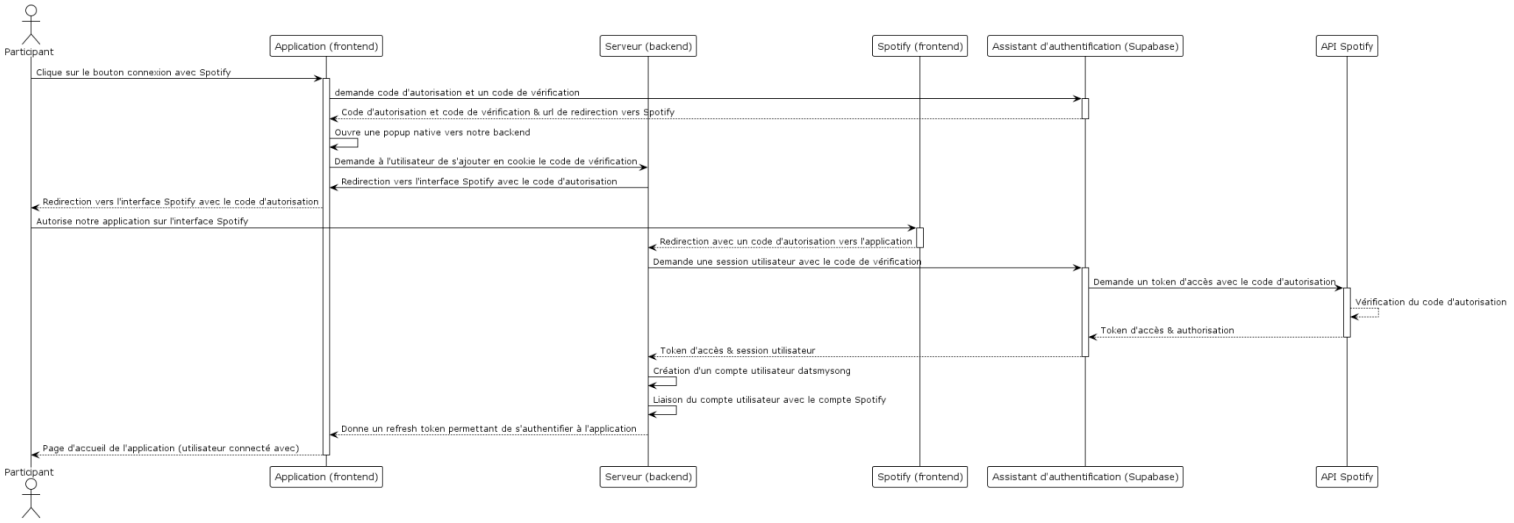
La tâche plutôt originale que je peux présenter est le flux Proof Key for Code Exchange (PKCE) pour l'authentification avec un provider. Le protocole PKCE est un flow d'authentification OAuth particulier. Nous l'avons mis en place dans un premier temps pour faire une authentification qui passe par notre serveur, pour récupérer notamment le token de notre provider (pour Spotify dans un premier temps), et créer l'utilisateur automatiquement lors de la première connexion.

Il s'est avéré avec nos recherches que de plus, ce protocole était une des implémentations les plus sûres pour l'authentification sur les applications mobiles. En effet, Expo nous permet de générer une application Web, mais aussi une application Android et une application iOS. Nos applications natives, sans back-end, peuvent utiliser un schéma d'URL personnalisé pour capturer les redirections (par exemple, ://datsmysong). Cette redirection peut être détournée par une application malveillante.

Cependant, l'implémentation à était assez longue, à cause du temps de compréhension du dispositif, et du manque de ressource en ligne avec nos technologies en particulier. Nous souhaitions quelque chose qui fonctionne sur mobile et sur Web. De plus, le flux est particulier car nous utilisons un service pour les providers et d'autres outils : Supabase.

J'ai pu perdre pas mal de temps à implémenter cette manière de faire, mais cette tâche étant un pilier de notre application car elle comprend le lien d'un compte Spotify avec un compte utilisateur de notre application, nous avons décidé de me faire passer du temps dessus. De plus ce flux se répète dans l'utilisation d'autres providers comme Google, très rapidement en itération 2 grâce à la logique déjà créé !

Première connexion à l'application via un compte Spotify flux PKCE



Objectifs à atteindre pour la fin du projet. Planning pour les deux itérations restantes.

Objectif à atteindre pour l'itération 6

Pour la fin de l'itération 6, nous avons pour objectif de pouvoir ajouter une musique avec une barre de recherche, cette tâche est technique que nous devons faire appel via api au service de streaming pour récupérer l'url de la musique cherché, avec une interface pour que l'utilisateur puisse sélectionner la bonne musique. De plus, nous devons aussi utiliser les api des différents services de streaming pour utiliser ses coups de coeurs / playlists des plateformes liées pour ajouter en file d'attente des musiques.
Nous avons d'autres petites tâches pour améliorer notre application et la tâche sur laquelle nous nous réservons de ne pas la commencer est la tâche du système d'amis. Nous aurons peut-être des imprévus tel que des bugs, et en itération 6, nous consacrerons possiblement plus notre temps à avoir une version très stable que de se lancer dans une grosse tâche comme celle-ci.

Cinquième itération

- Finir : Créer le menu de paramétrage d'une salle d'écoute
- Finir : Consulter l'historique et les détails d'une salle d'écoute passée
- Quitter une salle d'écoute en tant que participant
- Voter pour passer une musique (en cours, ou en file d'attente)
- Ajouter une musique via une barre de recherche, et des liens d'une autre plateforme
- Améliorer la page de création de salle

Sixième itération

- Actions de modération de la salle d'écoute
- Lier d'autres services de streaming, comme Deezer
- Rejoindre une salle d'écoute en tant qu'invité (via un lien)
- Système d'amis

- Utiliser ses coups de cœurs et playlists sur des plateformes de streaming pour ajouter des musiques en file d'attente

Conclusion

Tout au long des 4 premières itérations, nous avons suivi globalement notre étude préalable si l'on ignore les quelques modifications du schéma de la base de données. Comme dans tout projet, nous avons pu relever au cours du développement des problématiques par rapport au travail réalisé avant la première itération, tel que des oublis ou de légères modifications. Les plus gros changements se trouvent dans la planification des tâches, mais par l'utilisation organisée d'un Kanban directement intégré à GitHub, nous n'avons pas perdu de temps à se réorganiser.

Notre principale difficulté était d'obtenir une vélocité de développement satisfaisante.

En effet, chacun n'ayant pas le même niveau de connaissance des technologies utilisées et des paradigmes employés, il s'est avéré surtout lors des 3 premières itérations, que la vitesse de travail était relativement faible, ce qui explique le peu de tâches accomplies lors de ces itérations. En revanche, étant suivi par notre tuteur tout au long des 4 premières itérations, nous avons pris rendez-vous avec lui à la fin de l'itération 3 afin d'établir un plan d'action de mise en place d'un nouveau workflow nous permettant d'agir de façon plus rapide, en effectuant notamment des points journaliers chaque matin, et en dédiant une heure en fin de journée à la lecture du code d'autrui, permettant à chacun de s'améliorer, à la fois la personne relisant le code, qui consulte de ce fait du code écrit par d'autres, mais aussi la personne recevant les commentaires, puisqu'elle pourra améliorer son code et découvrir de nouvelles approches ou façon de faire par exemple.

Nous avons encore plusieurs tâches pour les deux dernières itérations, dont certaines importantes, mais nous avons une version de l'application contenant la totalité des briques nécessaires à son fonctionnement, ce qui devrait éviter de nous ralentir si l'on souhaite ajouter des fonctionnalités.

Nous sommes contents d'avoir pu délivrer un grand nombre de tâches à l'itération 4, ce qui démontre l'efficacité de notre nouveau workflow et ce qui est également causé par le déblocage de certaines tâches qui dépendaient d'autres tâches désormais finalisées.